

Non-mandatory exercises: F

Indexing and slicing lists

1. What will be printed?

```
tokenized_sent = ["The", "sun", "shines", "today"]
print(tokenized_sent[0])
print(tokenized_sent[2])
print(tokenized_sent[3])
print(tokenized_sent[-1])
```

the
shines
today
today

2. What will be printed?

```
pos_tags = ["DET", "NOUN", "VERB", "ADV"]
print(pos_tags[2:])
print(pos_tags[len(pos_tags) - 2:])
print(pos_tags[-2:])
print(pos_tags[:2])
print(pos_tags[:len(pos_tags) - 2])
print(pos_tags[:-2])
```

verb, adv
verb, adv
verb, adv
det, noun
det, noun
det, noun

Non-mandatory exercises: F

3. Given the following code as the first line:

```
cities = ["Stockholm",  
          "Hamburg",  
          "Berlin",  
          "Warsaw",  
          "Berlin",  
          "Copenhagen",  
          "Stockholm"]
```

Which five of the following lines will print 'Stockholm'?

- ☐ `print(cities[0])`
- ☐ `print(cities[1])`
- ☒ `print(cities[len(cities)-1])`
- ☒ `print(cities[-1])`
- ☐ `print(cities[-2])`

Non-mandatory exercises: F

Looping through lists

4. What will be printed?

```
def print_ing(tokenized_sentence):  
    for token in tokenized_sentence:  
        if token.endswith("ing"):  
            print(token)  
        else:  
            print(".")
```

```
words = ["fish",  
         "were",  
         "jumping",  
         "and",  
         "the",  
         "cotton",  
         "was",  
         "high"]
```

```
print_ing(words)
```

Non-mandatory exercises: F

5. What will be printed?

```
def get_re_prefix_words(words):  
    with_re_prefix = []  
    for word in words:  
        if word.startswith("re"):  
            with_re_prefix.append(word)  
    return with_re_prefix
```

[rebuild, reelect, reemerge]
class list

```
words = ["rebuild", "raccoon",  
         "radiate", "reelect",  
         "reemerge"]
```

```
re_candidates = get_re_prefix_words(words)  
print(re_candidates)  
print(type(re_candidates))
```

add
return genetives

6. What will be printed?

for now, it is returning,

```
def get_genetives(words):  
    genetives = []  
    for word in words:  
        if word.endswith("'s"):  
            genetives.append(word)
```

None
class NoneType

"/s"

```
tokenized = ["Mary's", "book", "was", "published", ".  
            "]
```

```
to_study = get_genetives(tokenized)  
print(to_study)  
print(type(to_study))
```

Actually, although the code does not return an error, it does not behave as the programmer had intended. The first `print()` line was meant to print `["Mary's"]`. Fix the code, so it works as intended.

Non-mandatory exercises: F

7. What will be printed?

```
COPULA = ["vara", "heta", "bliva", "kallas"]

def get_copula(words):
    found_copula = []
    for word in words:
        if word in COPULA:
            found_copula.append(word)
    return found_copula

tokenized = ["han", "vara", "frisk", "hel", "år",
             "men", "bliva", "förkyld", "i", "december"]

print(get_copula(tokenized))
```

[vara, bliva]